

PRACTICE EXERCISE 9: SERVICE - BROADCAST RECEIVER - NOTIFICATION

LEARNING OBJECTIVES

- Understand basic about Service. Types of Services and How to use Service in your application.
- Apply BroadcastReceiver in your application.
- Using Notification to display your application event

OVERVIEW

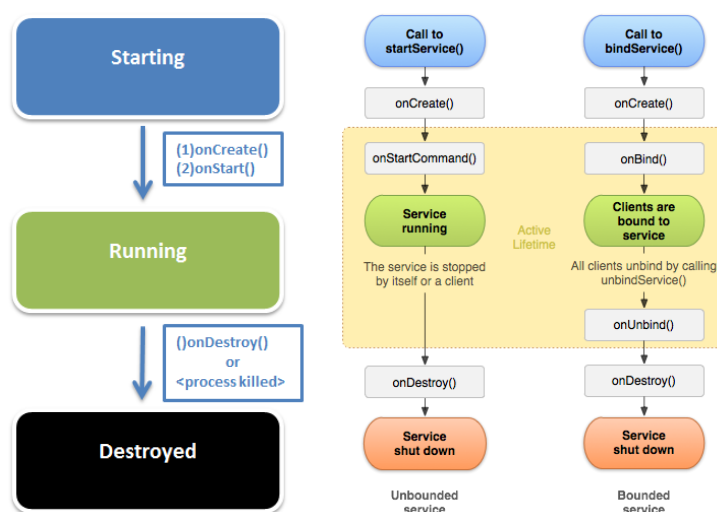
Service, Broadcast Receiver, and Notification are three essential components in Android app development:

Service: It's a background component without a user interface. It performs long-running tasks like playing music, downloading files, or processing data.

Broadcast Receiver: It's a component that listens for system-wide events or broadcasts. It allows your app to react to events even when it's not actively running.

Notification: It's a message displayed outside the app's UI to inform users about events or important information. Typically shown in the status bar or as pop-up alerts.

The android system force-stops a service only when it runs out of memory.



TERMINOLOGY

Background Service: A service that runs in the background without requiring direct user interaction. For example, playing music, downloading files.

Foreground Service: A service that runs in the background but has a user interface to make users aware of its operation. For example, music playback service.

Bound Service: A service that can be bound to other components in the application through intercommunication, such as Activity or Fragment.

IntentService: A service that runs in the background to handle queued tasks sequentially, suitable for tasks that do not require user interaction.

Service Lifecycle: The operating cycle of a service from its creation to its destruction.

onStartCommand(): The method called when a service is started by calling startService().

onBind(): The method called when another component wants to connect to the service by calling bindService().

onDestroy(): The method called when the service is destroyed.

Intent: An object representing an action or event that can be sent and received by different components in the system.

Sticky Broadcast: A type of broadcast that is retained after being sent so that newly registered apps can still receive it.

Local Broadcast: A broadcast sent and received within your own app and not shared with other apps.

System Broadcast: A broadcast sent from the Android system and can be received by various apps.

BroadcastReceiver: A component in Android that allows your app to listen for and respond to events or notifications.

Notification Channel: A mechanism that allows you to group related notifications together and provide separate settings for each group.

Notification Manager: A class in the Android SDK used to create and manage notifications.

NotificationCompat.Builder: A class in the Android Support Library used to build notifications compatible with different Android versions.

9.1. THEORETICAL SUMMARY

9.1.1. Knowledge

To effectively work with Service, Broadcast Receiver, and Notification in Android app development, you need the following knowledge:

Java or Kotlin: Basic knowledge of Java or Kotlin programming to understand and write code for various components of the app.

Basic Android Knowledge: Understanding of how the Android Framework works, interacting with different components of the system such as Activities, Fragments, Services, Broadcast Receivers, and Notifications.

Service: Understanding the concept of Service in Android and how to use them to perform background tasks.

Broadcast Receiver: Understanding how to create and register Broadcast Receivers to listen for events or notifications from the system or other apps.

Notification: Understanding how to create and manage notifications to inform users about events or important information.

Asynchronous Processing and Main Thread: Understanding how to handle asynchronous tasks in Android and manage the main thread and background threads.

Managing the Lifecycle of Android Components: Understanding the lifecycle of Android components such as Activity, Fragment, Service to manage and optimize these components in your app.

Knowledge of Data Storage Mechanisms: Understanding how to use data storage mechanisms such as SharedPreferences, SQLite Database to store data in Android apps.

9.1.2. Skill

To effectively work with Service, Broadcast Receiver, and Notification in Android app development, you need the following skills:

Java Programming: Strong understanding of Java programming to write code for various components of the app.

Understanding of Android Framework: Basic knowledge of how the Android Framework works, interacting with different components of the system such as Activities, Fragments, Services, Broadcast Receivers, and Notifications.

Asynchronous Processing: Understanding how to handle asynchronous tasks in Android and using mechanisms like AsyncTask or Kotlin Coroutines to manage them.

Managing the Lifecycle of Android Components: Understanding the lifecycle of Android components such as Activity, Fragment, Service to manage and optimize them.

Knowledge of Data Storage: Understanding of data storage mechanisms in Android such as SharedPreferences, SQLite Database for storing and accessing data.

Understanding of Inter-Component Communication: Knowledge of using Intent to communicate between components in the app such as invoking Service, sending and receiving Broadcasts.

Debugging and Troubleshooting Skills: Proficiency in debugging and troubleshooting in Android apps to identify and resolve issues that may arise during development.

9.2. BASIC PRACTICE

9.2.1. Exercise 1

Write a simple Android Notification application that includes a Service and a Broadcast Receiver. In this application, upon startup, a Notification will be sent from the Service and displayed on your device.

Please follow these steps:

Step 1: Create a New Project in Android Studio

Create new project with name : LabNine. (Should choose Empty Activity, don't need to use other layout templates)

Step 2: UX/UI Design:

Next, update activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/buttonNotification"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```

```

        android:text="@string/notification"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

Next, update strings.xml

```

<resources>
    <string name="app_name">LabNine</string>
    <string name="notification">Create Notification</string>
</resources>

```

Step 3: Handling event

Next, update MainActivity.java

```

package edu.uef.labnine;

import . . . ;

public class MainActivity extends AppCompatActivity {

    NotificationManager nManager;
    NotificationCompat.Builder mBuilder;
    Notification notify;
    Button bt_notification;
    String CHANNEL_ID = "mChannelId_message";
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        bt_notification= findViewById(R.id.buttonNotification);
        bt_notification.setOnClickListener(view -> {
            //Initial NotificationManager object

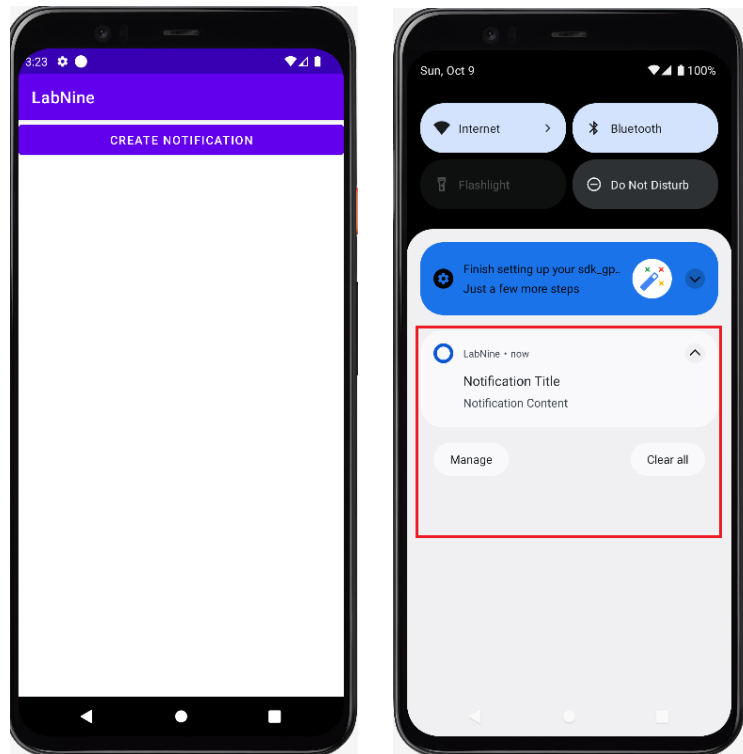
nManager=(NotificationManager)getSystemService(Context.NOTIFICATION_SERVICE);

            mBuilder=new
NotificationCompat.Builder(getApplicationContext(),CHANNEL_ID);
            mBuilder.setTitle("Notification Title");
            mBuilder.setText("Notification Content");
            mBuilder.setSmallIcon(R.mipmap.ic_launcher);
            mBuilder.setAutoCancel(true);

            //initialize the Notification with the build() function from the
Notification.Builder object
            notify=mBuilder.build();
            nManager.notify(0, notify);
        });
    }
}

```

Run your application and Swipe down to see Notification

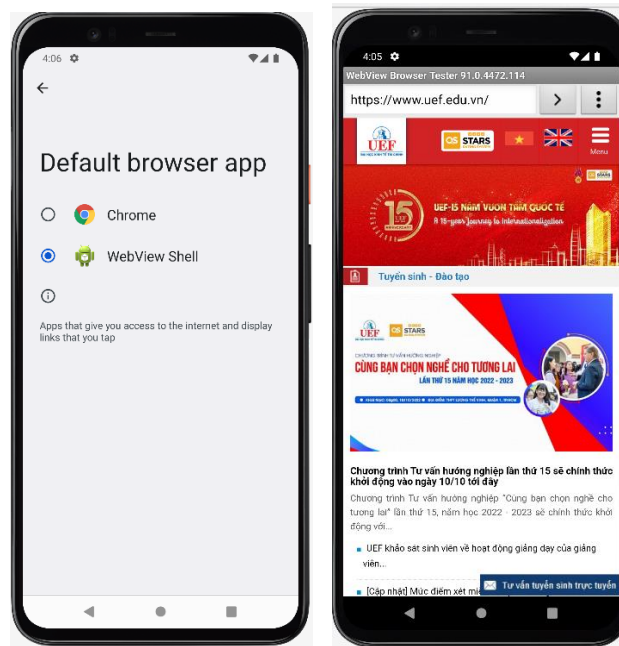


We want to surf <https://www.uef.edu.vn/> after click on Notification. Update function:

```
//Initial NotificationManager object
...
mBuilder.setAutoCancel(true);

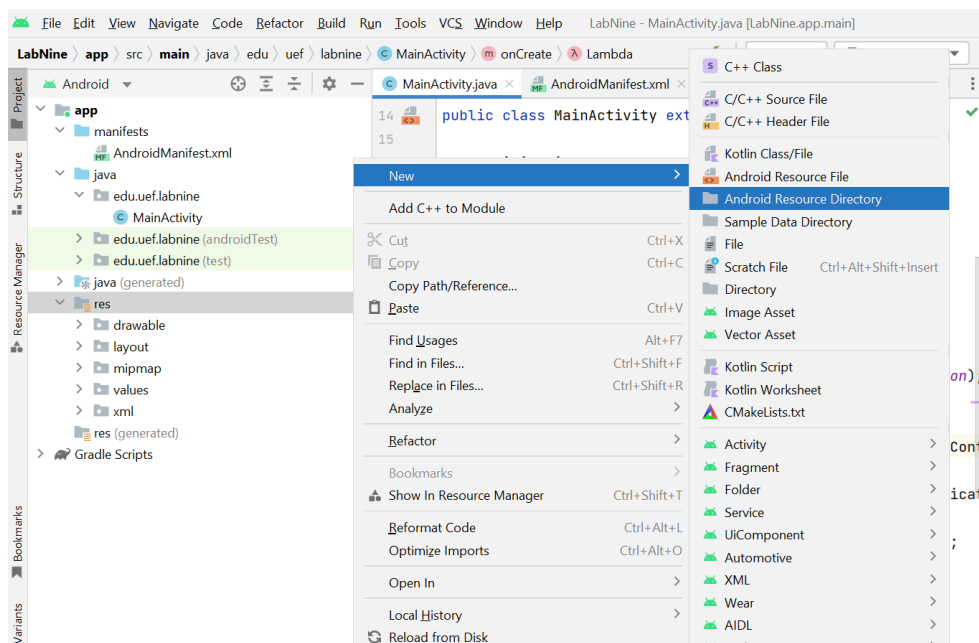
//Surf your website when click Notification
Intent i=new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.uef.edu.vn/"));
PendingIntent peIntent= PendingIntent.getActivity(getApplicationContext(),
    0, i, PendingIntent.FLAG_IMMUTABLE |
    PendingIntent.FLAG_UPDATE_CURRENT);
//Assign PendingIntent for Notification
mBuilder.setContentIntent(peIntent);
//initialize the Notification with the build() function from the
Notification.Builder object
notify=mBuilder.build();
nManager.notify(0, notify);
```

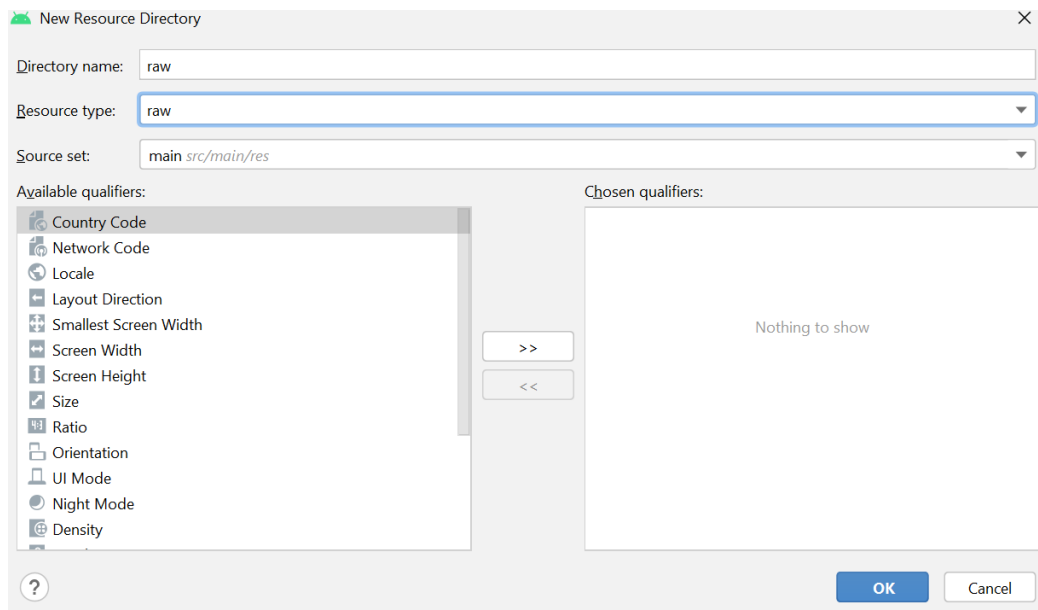
Change default browser of AVD to WebView. Run your application and Swipe down to see Notification. Click on Notification.



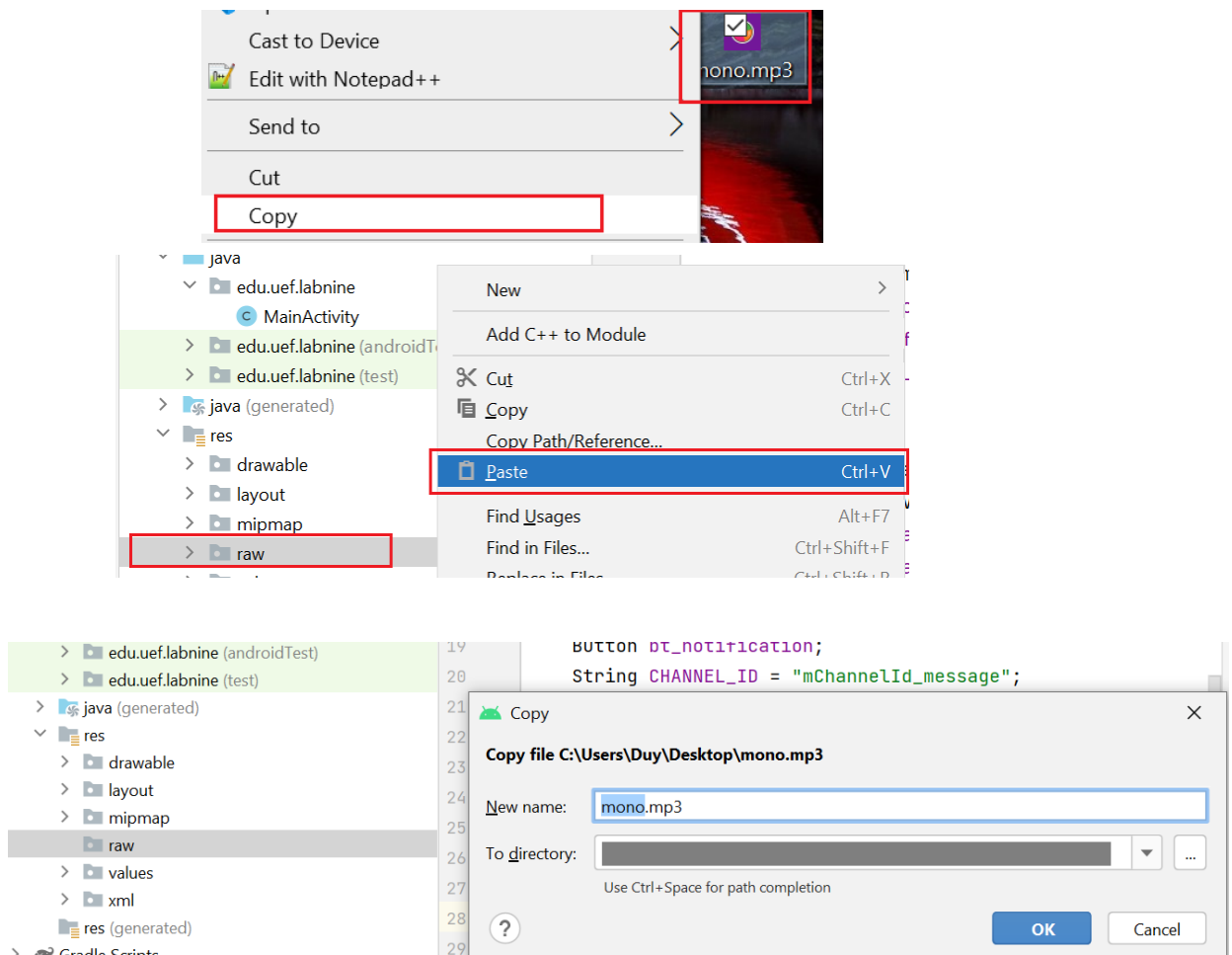
Create a service to play music on background (The music doesn't stop when exiting the app):

Add mp3 file





After creating the **raw** folder successfully go to your own system and copy the required audio files as we do normally. Now again come back to the android studio and right-click on the raw folder. Then click on the Paste button as shown in the below image.



Next, update Layout file (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    . . .

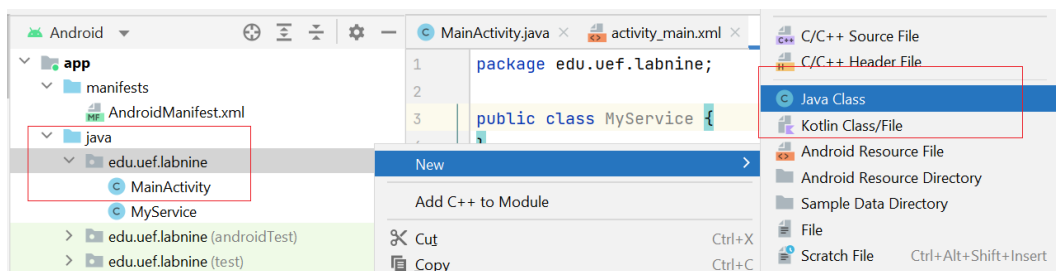
    <Button
        android:id="@+id/buttonStartMusic"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="@string/start"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/buttonNotification" />

    <Button
        android:id="@+id/buttonStopMusic"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="@string/stop"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/buttonStartMusic" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Next, update strings.xml

```
<resources>
    <string name="app_name">LabNine</string>
    <string name="notification">Create Notification</string>
    <string name="start">Start Music</string>
    <string name="stop">Stop Music</string>
</resources>
```

Next, create new java file "MyService.java"



Next, update AndroidManifest.xml

```
<application . . . >
    . . .
    <service android:enabled="true" android:name=".MyService"/>
</application>
```

Next, update **MyService.java**

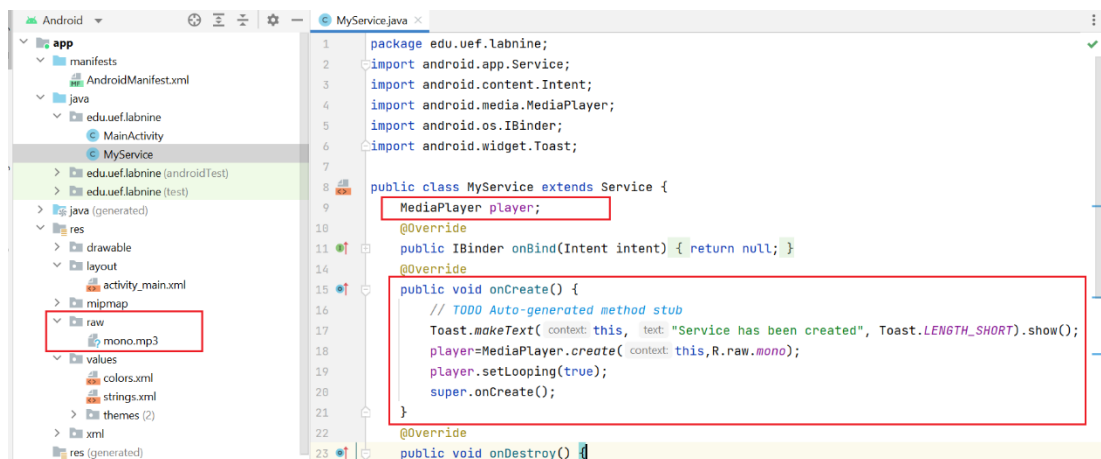
MyService class inherits from Service class and overrides the following functions:

```
package edu.uef.labnine;
import android.app.Service;
import android.content.Intent;
import android.os.IBinder;
public class MyService extends Service {
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
    @Override
    public void onCreate() {
        // TODO Auto-generated method stub
        super.onCreate();
    }
    @Override
    public void onDestroy() {
        // TODO Auto-generated method stub
        super.onDestroy();
    }
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        // TODO Auto-generated method stub
        return super.onStartCommand(intent, flags, startId);
    }
}
```

Next, declare global variables in MyService class:

```
MediaPlayer player;
```

Next, in the onCreate() function, create the **toast** and initialize the **player** object



Next, in the onStartCommand() function, add player.start(); to play music

```
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    // TODO Auto-generated method stub
    Toast.makeText(context: this, text: "Service is running", Toast.LENGTH_SHORT).show();
    player.start();
    return super.onStartCommand(intent, flags, startId);
}
```

Next, in the onDestroy() function

```
@Override
public void onDestroy() {
    // TODO Auto-generated method stub
    Toast.makeText(context: this, text: "service canceled", Toast.LENGTH_SHORT).show();
    player.stop();
    super.onDestroy();
}
```

Create a thread to count the number of signal and display it continuously with Toast to see that the service is running in the background. Update MyService class:

Next, declare global variables

```
MediaPlayer player;
public Handler mHandler=new Handler();
boolean isRunning=false;
Runnable mRunnable;
int count=0;
```

Next, create new function:

```
public void countSignal()
{
    if(isRunning)
    {
        mRunnable= () -> {
            // TODO Auto-generated method stub
            count++;
            Toast.makeText(getApplicationContext(), text: "Count: "+ count, Toast.LENGTH_SHORT).show();
            countSignal();
        };
        mHandler.postDelayed(mRunnable, delayMillis: 5000);
    }
}
```

Next , in the onStartCommand() function

```
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    // TODO Auto-generated method stub
    Toast.makeText(context: this, text: "Service is running", Toast.LENGTH_SHORT).show();
    player.start();
    isRunning=true;
    countSignal();

    return super.onStartCommand(intent, flags, startId);
}
```

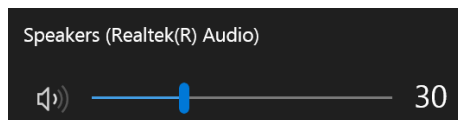
Next , in the onDestroy() function

```
@Override
public void onDestroy() {
    // TODO Auto-generated method stub
    Toast.makeText(context: this, text: "service canceled", Toast.LENGTH_SHORT).show();
    player.stop();
    mHandler.removeCallbacks(mRunnable);
    super.onDestroy();
}
```

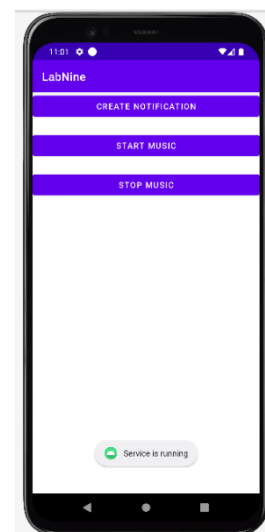
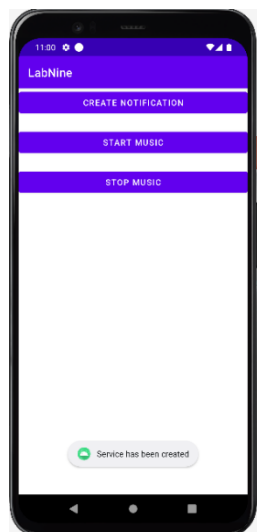
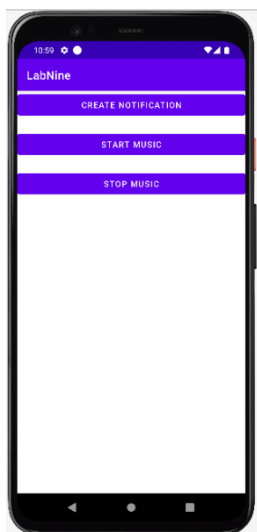
Next, update MainActivity.java

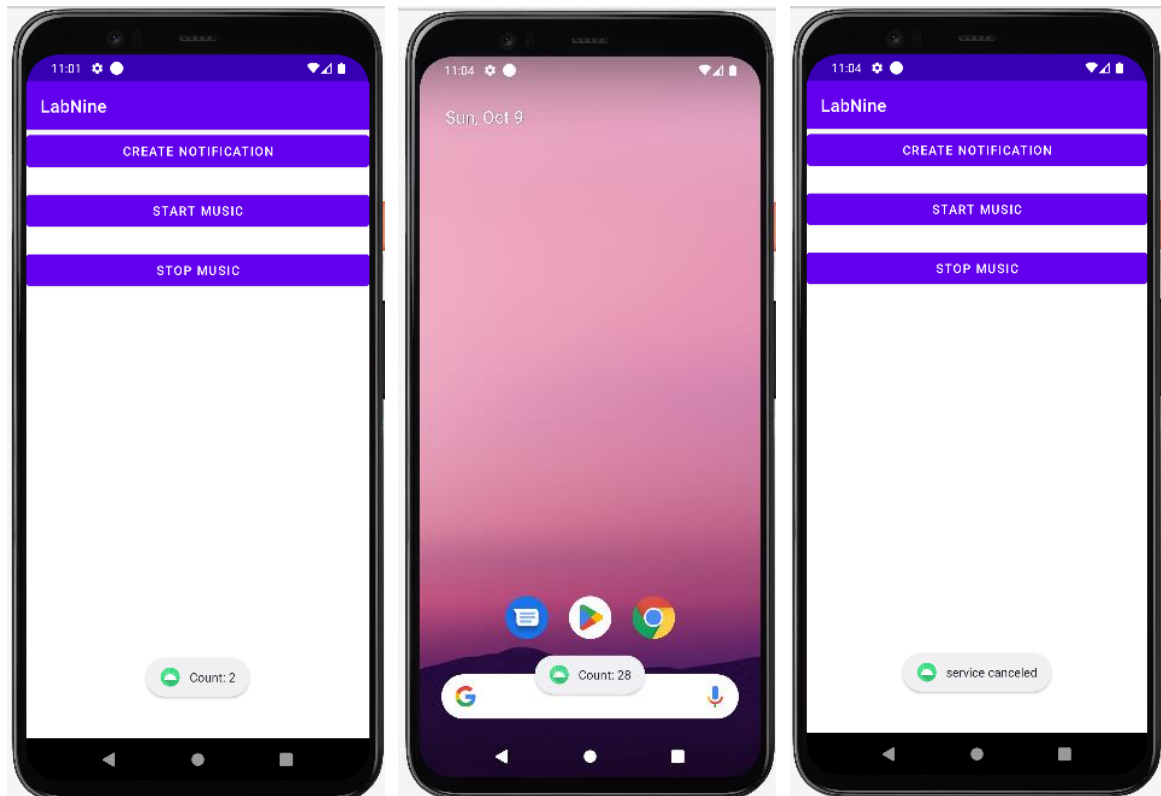
```
Button bt_start, bt_stop;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    bt_start = findViewById(R.id.buttonStartMusic);
    bt_stop = findViewById(R.id.buttonStopMusic);
    . . .
    bt_start.setOnClickListener(view -> {
        startService(new Intent(getApplicationContext(), MyService.class));
    });
    bt_stop.setOnClickListener(view -> {
        stopService(new Intent(getApplicationContext(), MyService.class));
    });
}
```

Increase your computer volume



Run your project and click Start music. Exit your application. Open app again and click Stop music button.

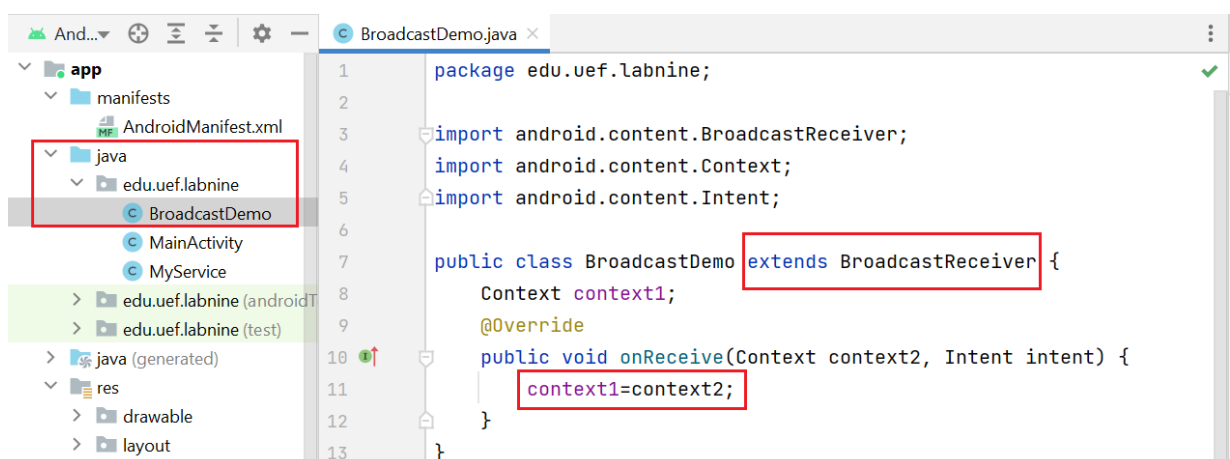




Testing the Notification function when there is an incoming call using **Broadcast Receiver**

Create new Java file ***"BroadcastDemo.java"***

Make MainActivity class inherit from BroadcastReceiver and add the onReceive() function. Declare global variable: "Context" and in onReceive() assign value to Context variable.



Continues to update onReceive() and Build an Internal Class named MyPhoneStateListener

```

BroadcastDemo.java x AndroidManifest.xml x
8 import android.telephony.TelephonyManager;
9 import android.widget.Toast;
10
11 public class BroadcastDemo extends BroadcastReceiver {
12     Context context1;
13     @SuppressWarnings("UnsafeProtectedBroadcastReceiver")
14     @Override
15     public void onReceive(Context context2, Intent intent) {
16         context1=context2;
17         // Register TELEPHONY MANAGER to listen to event
18         TelephonyManager mgr = (TelephonyManager) context2
19             .getSystemService(Context.TELEPHONY_SERVICE);
20         //Create Listener
21         MyPhoneStateListener PhoneListener = new MyPhoneStateListener();
22         // Register listener with LISTEN_CALL_STATE
23         mgr.listen(PhoneListener, PhoneStateListener.LISTEN_CALL_STATE);
24     }
25     private class MyPhoneStateListener extends PhoneStateListener {
26         public void onCallStateChanged(int state, String incomingNumber) {
27             // Log.d("MyPhoneListener",state+" incoming no:"+incomingNumber);
28             // state = 1 means when phone is ringing
29             if (state == 1) {
30                 String msg = " New Phone Call Event. Incoming Number : "+incomingNumber;
31                 int duration = Toast.LENGTH_LONG;
32                 Toast toast = Toast.makeText(context1, msg, duration);
33                 toast.show();
34             }
35         }
36     }
37 }

```

Grant permission in **AndroidManifest.xml**. Remove the Activity tags and contents (the app doesn't need the Activity).

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.LabNine"
        tools:targetApi="31">

        <!--<activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

            <meta-data

```

```

        android:name="android.app.Lib_name"
        android:value="" />
    </activity>
    <service android:enabled="true" android:name=".MyService"/>-->
    <receiver
        android:name=".BroadcastDemo"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.PHONE_STATE" />
        </intent-filter>
    </receiver>
</application>

</manifest>

```

Run your application. Because there is no activity, the application does not display the interface. Look at Logcat for the application in action.

9.2.2. Exercise 2

Developing an application **“Daily Reminder App”**, Including requirements:

- Create a Service to check daily events that need reminders (e.g., friends' birthdays, tasks to complete, etc.).
- Use a Broadcast Receiver to receive information from the Service when there are events that need reminders.
- Display Notification messages to users when there are events that need reminders.

Please follow these steps:

Step 1: Create a New Project in Android Studio

Create new project with name : **laNineAdv**. (Should choose Empty Activity, don't need to use other layout templates)

Step 2: UX/UI Design

```

activity_main.xml (Layout file for MainActivity)
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <TimePicker
        android:id="@+id/timePicker"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:timePickerMode="spinner" />

```

```
<Button
    android:id="@+id/setReminderButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/timePicker"
    android:layout_centerHorizontal="true"
    android:text="Set Reminder" />
</RelativeLayout>
```

Step 3: Handling events

MainActivity.java (Entry point of the application):

```
import android.app.AlarmManager;
import android.app.PendingIntent;
import android.content.Context;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TimePicker;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import java.util.Calendar;

public class MainActivity extends AppCompatActivity {

    private TimePicker timePicker;
    private Button setReminderButton;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        timePicker = findViewById(R.id.timePicker);
        setReminderButton = findViewById(R.id.setReminderButton);

        setReminderButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                int hour = timePicker.getCurrentHour();
                int minute = timePicker.getCurrentMinute();

                Calendar calendar = Calendar.getInstance();
                calendar.set(Calendar.HOUR_OF_DAY, hour);
                calendar.set(Calendar.MINUTE, minute);
```



```

        calendar.set(Calendar.SECOND, 0);

        setAlarm(calendar);
    }
});
}

private void setAlarm(Calendar calendar) {
    AlarmManager alarmManager = (AlarmManager)
getSystemService(Context.ALARM_SERVICE);
    Intent intent = new Intent(this, ReminderReceiver.class);
    PendingIntent pendingIntent = PendingIntent.getBroadcast(this, 0,
intent, 0);

    if (calendar.before(Calendar.getInstance())) {
        calendar.add(Calendar.DATE, 1);
    }

    alarmManager.setExact(AlarmManager.RTC_WAKEUP,
calendar.getTimeInMillis(), pendingIntent);
    Toast.makeText(this, "Reminder set successfully!",
Toast.LENGTH_SHORT).show();
}
}

```

9.3. APPLICATION EXERCISES

1. Battery Status App:

- Develop a Service to monitor the status of the device's battery (e.g., current battery level, charging status, etc.).
- Utilize a Broadcast Receiver to receive battery status information from the Service.
- Implement Notification messages when the battery level is low or when the device is charging.

2. Network Connection Checker App:

- Create a Service to continuously check the network connection status of the device.
- Use a Broadcast Receiver to receive information from the Service when the network connection status changes.
- Display Notification messages when the network connection changes from connected to disconnected and vice versa.

3. Background Task Manager App:

- Develop a Service to manage background tasks of the application (e.g., data downloading, data synchronization, etc.).
- Utilize a Broadcast Receiver to receive requests from the application or from the system.
- Implement Notification messages to report the progress of background tasks being executed.

REFERENCES

- [1]. Android Developers. Service. Retrieved from <https://developer.android.com/guide/components/services>
- [2]. Android Developers. Broadcasts Overview. Retrieved from <https://developer.android.com/guide/components/broadcasts>
- [3]. Android Developers. Notifications Overview. Retrieved from <https://developer.android.com/guide/topics/ui/notifiers/notifications>
- [4]. Coursera. Android App Development Specialization. Retrieved from <https://www.coursera.org/specializations/android-app-development>
- [5]. Griffiths, D., & Griffiths, D. Head First Android Development: A Brain-Friendly Guide. Publisher.

REFERENCES

1. Phillips, B., & Stewart, C. (2017). Android Programming: The Big Nerd Ranch Guide. Big Nerd Ranch
2. Griffiths, D., & Griffiths, D. (2017). Head First Android Development. O'Reilly Media.
3. Horton, J. (2016). Android Programming for Beginners. Packt Publishing.
4. Meier, R. (2018). Professional Android (4th ed.). Wrox.
5. Vogel, L. (2016). Android 6 for Programmers: An App-Driven Approach (3rd ed.). Pearson.
6. Darcey, L., & Conder, S. (2017). Android Wireless Application Development Volume I: Android Essentials (4th ed.). Addison-Wesley Professional.
7. Conder, S., Darcey, L., & Sansing, J. (2015). Android Wireless Application Development Volume II: Advanced Topics (3rd ed.). Addison-Wesley Professional.
8. Silva, N. (2019). Mastering Android Development with Kotlin: Deep dive into the world of Android to create robust applications with Kotlin. Packt Publishing.
9. Morris, S. (2019). Kotlin Programming: The Big Nerd Ranch Guide. Big Nerd Ranch.